Tight-Inclusion CCD AI 自动优化阶段汇报

报告日期：2026 年 10 月 15 日 | 最新全量实验：2026 年 10 月 10 日

本阶段围绕连续碰撞检测的计算效率开展 AI 辅助优化。最终 C20 在 59,387,883 条查询上的核心计算耗时由 72.746 秒降至 32.992 秒，降低 54.65%，约为原版的 2.205 倍速度。四组漏检均为 0，误报由 312 个降至 191 个。[S1]

最初“耗时降低至少 20%”的目标已经达到；随后提高到 80% 的目标尚未达到。研究完成 20 个有效候选后，按约定上限停止。最新成果是经过全量验证的组合优化版本，以及可追溯的代码、实验与数值论证记录。[S2–S4]

0. 跑了什么问题

问题：让同一套碰撞检测任务更快

CCD（连续碰撞检测）判断几何体在一段运动过程中是否发生碰撞，并返回碰撞时间 TOI。本次研究对象是 Tight-Inclusion：针对线性运动的点与三角形（VF）、边与边（EE）查询，优化其 CPU 单线程执行效率。

AI 在这里承担提出优化假设、修改 C++ 实现、执行评测和整理证据的工作；最终运行和被计时的是几何算法程序。本阶段没有训练新的神经网络，也没有用模型预测替代碰撞判定。

固定约束与评价标准

使用完整发布数据中的全部 59,387,883 条查询；保持 double 精度、BFS、距离容差 1e-6、最大迭代数 1e6、minimum_separation=0。要求四组漏检均为 0、各组误报不超过原版，并保留保守性与碰撞时间行为。

指标是 CCD 核心计算耗时：包含 API 参数转换与 CCD 调用，排除数据读取、解析、缓存转换、结果比较和写盘。54.65% 不能直接用于描述整个应用程序的端到端提速。历史基线 74.37439 秒仅作参考，最终比较使用同期原版。

实验环境：Intel Core Ultra 7 265KF / Windows 11 / MSVC 19.44 Release；单线程绑定逻辑 CPU 0。人工组查询数与论文表格不同，不能称为逐项复现论文全部数字。[S1,S2]

0. 中间经历了什么过程

从固定评测规则开始，逐个验证假设

先保留原 baseline 工作树与数据，另建研究工作树和候选评测入口。筛选集预先固定为 546,337 条，包含全部人工查询、全部仿真真碰撞，以及每个仿真文件每 128 条抽样。每个候选保存假设、代码版本、构建参数、正确性结果与计时日志。

候选先过定向检查和子集筛选，再对有潜力的版本做全量测试。关键阶段让原版与候选交替运行，各做三次，以全量核心耗时中位数比较。最终累计 20 个有效候选；构建失败、重复运行和启动失败不额外计数。[S2]

阶段	同期原版 / 秒	候选 / 秒	降耗
C02：复用插值、前移初筛	72.602	52.472	27.73%
C05：算术与初始化优化	72.108	45.378	37.07%
C07：增加 swept AABB	72.727	42.485	41.58%
C20：最终组合版本	72.746	32.992	54.65%

各行是对应阶段的三轮全量中位数，不是同一次消融实验。不能把阶段差值直接解释为某个模块的独立贡献。[S1,S3,S4]

优化路线如何演进

C01–C06 主要减少重复插值、除法、初始盒准备和函数调用；C07 加入运动包围盒过滤。C08–C16 进一步引入 Bernstein 共面性证书、投影分离证书，并调整调用时机、压缩节点和减少分配。C19 优化误差界求值；C20 复用搜索队列容量。

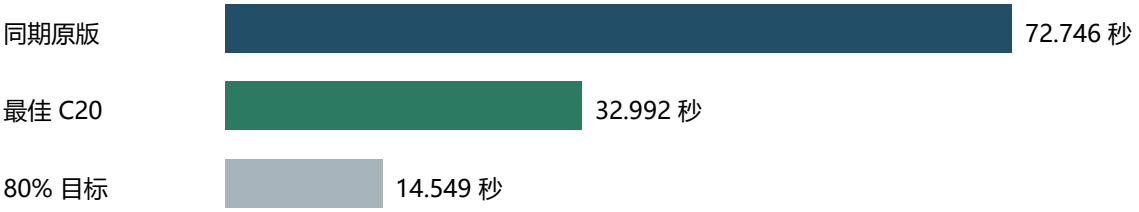
没有保留的尝试同样提供了信息

C08 的证书在子集上表现较好，但全量单轮耗时达到 67.502 秒，说明对大量简单查询统一增加证书成本并不合算。C09 改为搜索计数达到 refine==32 时才尝试证书；该计数不等同于每条查询做了 32 次几何二分。

C12 的显式 AVX2 版本在四组子集上均变慢；C17 的分段投影虽多消除 11 个误报，但额外计算成本过高；C18 未证明收益，均回退。C10 单独表现变慢，只作为后续内联组合的一部分保留。由此可见，“听起来更高级”不能替代实测。[S2,S5]

过程中曾发生 Windows 代码完整性策略阻止程序启动；失败记录被保留，后续恢复后完成测试。最终记录没有待执行候选或仍在运行的实验。[S6]

0. 当前最新结果与验证边界



轮次	原版 / 秒	C20 / 秒
1	72.745920	32.749760
2	72.927355	32.992082
3	72.412401	33.248423
中位数	72.745920	32.992082

执行顺序为原版 1、C20 1、原版 2、C20 2、原版 3、C20 3。每轮先将四组时间求和，再对三轮总时间取中位数；降低比例为 1−32.992/72.746。[S1]

组别	查询数	原版误报	C20 误报	C20 漏检
仿真点面 VF	17,959,962	2	2	0
仿真边边 EE	41,372,584	17	6	0
人工点面 VF	21,208	90	53	0
人工边边 EE	34,129	203	130	0
合计	59,387,883	312	191	0

三次 C20 的正确性计数完全一致。误报减少 121 个，即误报数量下降 38.78%；这不是全体查询错误率的下降比例。四组各自耗时中位数的降幅为 66.78%、56.59%、42.58%、50.59%。[S1,S5]

正确性证据覆盖到哪里

每条查询比较 hit、TOI、output_tolerance 的完整 24 字节输出。只允许有布尔真值支持的原版误报被剔除：每轮恰好 121 条；其余 59,387,762 条输出逐位一致，包括所有保留碰撞的 TOI。

数据没有精确 TOI 真值，因此这里验证的是与原版保留碰撞输出一致，尚未独立测量绝对碰撞时间误差。距离容差 1e-6 不能解释成时间误差上限；测试集上的零漏检也不能单独替代一般保守性证明。

最终核心提交：0c1f229b4e10a3ad95033956f6604f5740c64c8d。报告生成时再次核对当前 src/ 与该版本无差异。[S2]

1. 接下来准备怎么做

建议把下一阶段目标设为：先解释清楚当前收益，再针对困难查询继续优化。以下是待开展的研究计划，尚不属于已完成实验。

第一步：补消融，回答“为什么快”

以 C20 为固定对照，先构建原版、仅实现优化、仅几何过滤、完整 C20 四类配置，再在 C20 中逐项关闭关键过滤与队列复用。涉及依赖的模块按组合比较，明确交互效应。除核心时间外，同时记录过滤调用次数、成功率、过滤自身耗时、BFS 节点数及峰值队列。

交付目标是一张贡献表：哪些改动减少了单次计算成本，哪些减少了搜索工作，哪些收益依赖组合。保持全部正确性门槛，最终有潜力的配置采用同期交替全量复测。

第二步：在 C20 上重新定位困难查询

人工查询只有 55,337 条，占全量约 0.0932%，却占 C20 三轮核心时间的 50.08%–50.73%。先按单条耗时、碰撞真值和 VF/EE 类型排序，分析近共面、擦边接触、退化构型与大量细分等情况；这些类别是待验证的解释，不能预先当作已确认原因。[S5]

重点尝试自适应证书调用时机、针对困难阴性查询的更紧保守界，以及能减少无效细分的策略。新判定必须先说明保守性；不能证明时保留原求解器回退。旧 C09 的热点诊断只能帮助选方向，不能代替 C20 的重新剖析。

第三步：检验收益能否推广

增加未参与候选选择的独立场景与定向边界案例，复查尺度变化、近接触和退化输入。对保留碰撞继续比较原版 TOI；如需主张绝对时间精度，应另建高精度参考与误差评价。跨编译器或平台复测，并单独报告内存和端到端开销。

集成时还要关注结构布局改变带来的 ABI 重新编译要求，以及队列容量复用约每线程 7.5 MiB 的保留内存上界。现有证明依赖规定浮点环境，推广时需要重新核验。[S5]

第四步：重新设置可验收目标

建议先以相对 C20 再降低 15%–20% 为探索目标，即约 26.39–28.04 秒；零漏检保持，各组误报不超过 C20 的 2/6/53/130。这是建议目标，不是性能预测。原“相对原版降低 80%”对应 14.549 秒，仍需比 C20 再降 55.90%。

按当前耗时构成估算，即使把人工组成本降到零，剩余时间仍约 16.5 秒，高于 14.55 秒。因此继续追求 80% 时，还必须优化仿真组；仅攻克人工组不足以达标。下一阶段应另行明确候选预算和停止条件。

汇报讲稿与证据索引

可直接口述的简版

“这次我研究的是 Tight-Inclusion 连续碰撞检测的速度优化。任务是在不降低精度、不引入漏检、误报不超过原版的前提下，让相同的近六千万条查询运行得更快。AI 负责提出假设、修改实现和组织实验，我通过固定评价规则与保存的实验记录来检查产出。

整个过程先建立独立评测环境，再用固定子集筛选，最后让原版和候选交替做全量测试。最初降低 20% 的目标在 C02 达到；后来把目标提高到 80%，最终完成了 20 个候选。最新 C20 的核心耗时由 72.746 秒降至 32.992 秒，降低 54.65%，四组都没有漏检，误报也从 312 个减到了 191 个。80% 的目标还没有达到。

下一步，我准备先做消融，弄清各项优化真正贡献了什么，再分析当前仍然很慢的查询。人工构造查询不到千分之一，却占了一半左右计算时间，是优先研究对象。同时补充独立数据与数值精度验证，让当前的速度结果进一步变成能够解释、能够推广的研究结论。”

核查依据

以下文件均位于 Tight-Inclusion-autoresearch 工作树；S1 为主要原始结果入口，报告中的事实、解释与建议分别标明用途。

[S1] 最终全量结果与六轮原始记录
research-results/final80-c20-summary.json；
research-results/runs/final80-c20-{reference,candidate}-{1,2,3}/results.json

[S2] 20 个候选、最终源码与完整性审计
research-results/final80-completion-audit.json；
research/FINAL_RESULTS80.rst

[S3] 20% 初始目标阶段
research/RESULTS.rst；
research-results/final-c02-summary.json

[S4] 中间里程碑
research-results/phase80-c05-summary.json；
research-results/phase80-c07-summary.json

[S5] 优化解释、分组数据与数值依据
research/C20_CHANGE_ANALYSIS.rst；
research-results/c20-analysis-metrics.json；
research/NUMERICS.rst；
research/CONTINUED_NUMERICS.rst

[S6] 当前停止状态与历史环境问题
AUTORESEARCH.rst；
research/NEXT_STEPS.rst

阅读顺序建议：先看 S1 确认收益与正确性，再看 S2 核对版本和候选记录，最后结合 S5 理解方法及其适用条件。复现命令、源码归档与补丁位置见 S2 中的最终报告。